



AI governance with Saidot

The product today, and the road to enforcement

Strawman for discussion

September 2026

What we will cover

- 01 Saidot today** one graph, one library, one system of record
- 02 Agentic use** the API and the MCP servers
- 03 Two camps** semantics on one side, enforcement on the other
- 04 The layer model** governance, policy as code, runtime, compliance
- 05 Roadmap** what ships, what is proposed, what is open

Saidot today

Govern AI systems, models and agents in one connected graph.

Saidot

THE PROBLEM

Governance cannot keep pace with AI in production

1

No single view of AI systems, models and agents

Models, agents, datasets and suppliers are tracked separately. Nobody can say which models power which products, or what breaks when one is swapped.

2

Every framework governed separately

EU AI Act, ISO 42001, NIST AI RMF and internal policy are each mapped by hand, then evidenced again. Every new jurisdiction restarts the work.

3

Governance has become the bottleneck

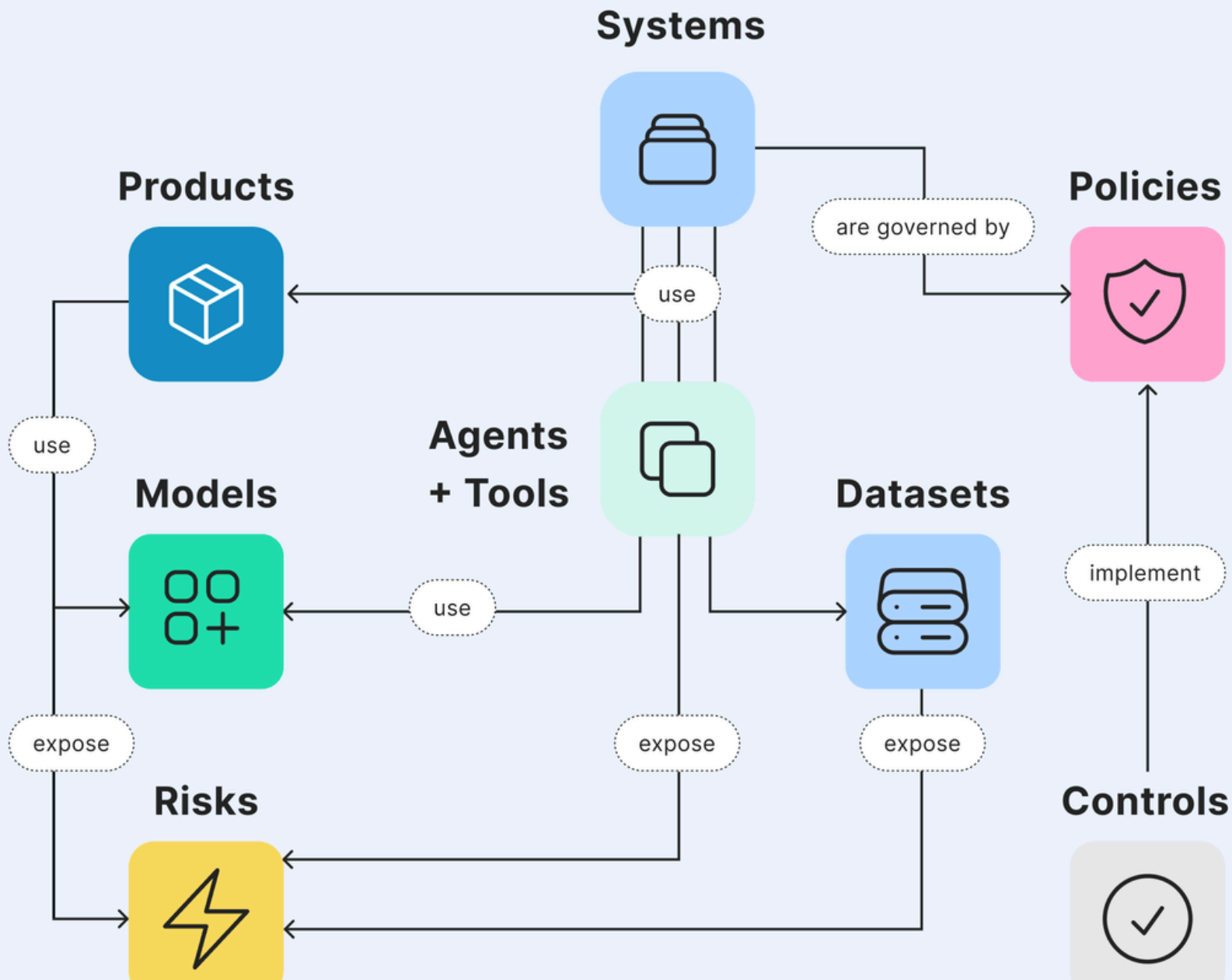
A mandate to move faster collides with manual review. Data scientists meet governance as a brake.

One graph

The Saidot graph connects products, systems, models, agents, tools, datasets, risks, policies, controls, evidence and incidents.

Govern once, inherit everywhere. A control set on a model reaches every system that uses it, and a risk exposed by a dataset reaches every agent reading from it.





The library

260+

risks

620+

controls

110+

policies

170+

third-party AI models

Curated and versioned by Saidot. Nobody starts AI governance from a blank page.

What sets Saidot apart

1

Graph-native governance

Built on a graph, not on questionnaires. Scales to thousands of systems and inventories past fifty thousand entries, where form-based tools collapse under volume.

2

Expertise productised in the library

Software bundled with regulatory knowledge, delivered as a continuously updated service.

3

Integrated into AI infrastructure

MCP servers and APIs connect runtimes to governance: set controls, collect evidence, orchestrate approvals. The system of record for AI governance, inside the AI infrastructure.

Agentic use

95% of the platform is operable through the API.
The MCP servers put it in front of an agent.

MCP servers

Governance MCP

AI inventory and catalogue management: create, read and update systems, models, agents, datasets, risks and controls.

Library MCP

Read access to the curated library: policies, risks, models, products, evaluations and controls.

Docs MCP

Governance documentation and methodology guidance.

Cloud hosted over HTTP for Claude Desktop, VS Code, Cursor and other MCP clients. Docker over stdio for local or on-premise use, alongside the IDE.

What an agent does today

1

Register

an agent registers a system, its models and its datasets from the deployment it sees

2

Classify

rules propose a tier from the properties on record; a person ratifies it

3

Inherit

the tier resolves its control set through the graph

4

Evidence

the agent attaches evidence and opens the approvals the lifecycle asks for

5

Report

the state of every control is queryable, by a person or by the next agent



Governance flows down, runtime signals flow back

Runtime enforcement tools police individual agent actions in the moment.

Saidot is the governance layer above: the graph that defines the policies, risks and controls worth enforcing, and the evidence proving them.

We enable enforcement. We do not sit in the hot path.

Two camps

Two communities solve the same problem and barely read each other.

Semantics on one side, enforcement on the other

Semantic Web

STACK

RDF, SPARQL, ontologies: DPV, VAIR, AIRO

GOOD AT

Describing risks and rights machine-readably, linking messy vocabularies

MISSING

Execution. Nothing evaluates against a running system

WHO RUNS IT

Research groups, standards bodies

Policy as code

STACK

OPA and Rego, OSCAL, CI/CD gates

GOOD AT

Enforcing and evaluating against a live system

MISSING

Regulatory semantics

WHO RUNS IT

Security teams, already in production

The same premise on both sides

Prose does not scale

Obligations need a structured intermediate form that machines process and people review, on standard vocabularies.

Provenance is evidence

Their graph traces a risk to an incident.
Our record traces a control decision to the obligation, the property values checked and the rule version. An assertion without a trail is not evidence.

Their output is our input

A completed assessment is, to us, an event: who, when, attached evidence, expiry. What happens inside it is their domain.

A risk described in VAIR does not block a deployment or create an audit artefact. The missing piece is the runtime, and that is the bridge to build.

Two points of conflict

Automated classification is a coin toss

Their evaluation found LLM classification of the employment domain matched human experts about as often as chance, across two models. The AI Act risk tier must be a human-ratified property with provenance and expiry. Rules may suggest a tier. A person confirms it, because that one property scopes every downstream obligation.

The ontologies exist

DPV, VAIR and AIRO are published, versioned, and have carried AI Act semantics for years. The accurate claim is narrower: they are not integrated into any working compliance pipeline. That is a better pitch, because it is true.



This limits what can honestly be promised on automation. The customers-and-law side of that trade is the question for this room.

The layer model

Governance and compliance are separate layers, and a loop joins them.

Four layers, one loop

Governance

the human side: strategy, risk appetite, custom policies, tiering, scoping, thresholds

Policy as code

obligations mapped to properties and thresholds, compiled into an assessment policy and an enforcement policy

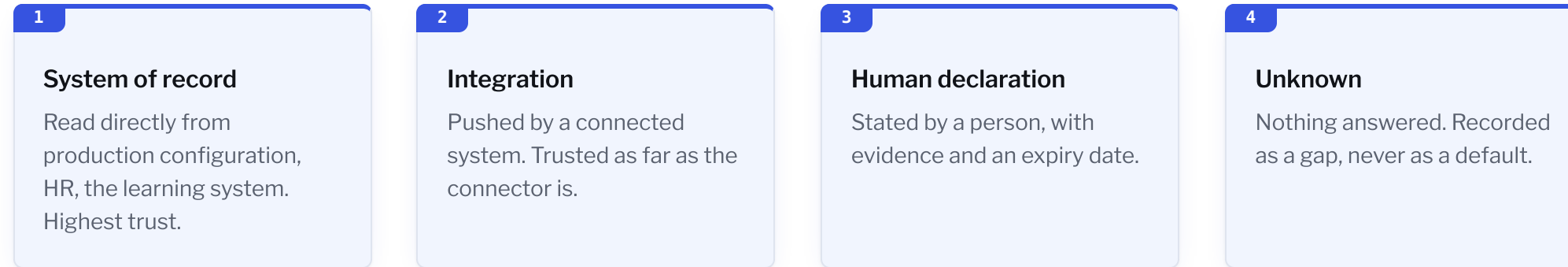
Runtime

enforces inline, reports state, discovers what is actually deployed

Compliance

evaluates to pass, fail or unknown, emits evidence and remediation items, and loops back to governance to change the system or change the rule

A property is a fact with a subject, a source and a trust level



Process obligations are not a separate class. "Staff trained" has a learning system behind it, "DPIA done" may be a declaration. The split is whether a system of record exists, which turns the gap list into a connector roadmap. Claims are events with expiry, never booleans: a boolean that cannot go stale cannot be audited.

OSCAL as the neutral format

One catalogue

AI, information security and privacy controls merged in one library, so the same evidence is never collected twice.

Crosses boundaries

Import and export in a format auditors and other tools already read, not a document export bolted on at the end.

Neutral language

Controls stated once, in neutral terms, with a mapper translating them into what a given runtime can check and emitting Rego for it.

Applicability and enforcement

Scoping rules

Does this control family apply to this system at all?

Tailoring rules

Does this specific control apply, given the system's own properties?

Coverage

Rules plus a coverage list, and the statement of applicability generates itself per system.



A runtime that cannot check a control returns unknown and routes the question to the learning system or to a person. If nothing answers, it stays a gap, and the gap says: find out.

What is actually new

1

Applicability per system instance

Computed from the instance's own properties. Cross-framework mapping at the organisation level already exists at Vanta and Drata. Per system, it does not.

2

Capability negotiation

A runtime explicitly replies that it cannot evaluate a check. The unknown persists as a gap until something else answers, instead of a silent pass or fail.

Saidot already holds the systems and the workflows that assign controls. That is why these two lead.

Roadmap

Shipped, in development, proposed.

Saidot

From governance to enforcement

Shipped

Governance, Library and Docs MCP servers. REST API over most of the platform. Observability API.

In development

Orchestrator MCP. Workflow automation: classification, approvals, reassessment triggers, evidence capture. Customisation of lifecycles, roles, thresholds and templates per organisation.

Proposed

Properties with source and trust level. OSCAL import and export. Applicability rules per system. Policy compiled per runtime, with capability negotiation and unknown as a first-class outcome.

The proposed column is this strawman, not a commitment.

Still open

Rule precedence

Two rules conflict on one control.
XACML's combining algorithms are the candidate, not yet chosen.

Coverage decomposition

How a unified control decomposes into what each runtime can attest to.

Point in time

Auditors ask what the state was on a given date. Data is mutated today, not appended, so last Tuesday cannot be reconstructed. The primitives exist. The design does not.

Two questions for this room

1

Pass, fail, unknown

Does a three-outcome model survive contact with a real auditor?

2

Applicability rules

Can they carry real legal nuance, or do they collapse at the first genuinely ambiguous article?



A strawman, not a position

Poke holes in it.